DxPS

Real-time Provisioning System

Installation and Running Guide

Windows 10/11 - Ubuntu 22.04/24.04 - macOS 13+

4	3	6 + 9	127.0.0.1:8088
steps to a running stack	platforms	services + simulators	dashboard

Item	Value
Document	DXPS-INSTALL-GUIDE
Version	1.2
Date	17 October 2026
Applies to	DxPS release 1.0.0 with the DxPS console; Go 1.26.6, PostgreSQL 18, Apache Kafka 4.3.1, Node.js 22

Table of Contents

1. What you are installing 3
2. Requirements 3
3. Package contents 4
4. Windows 10/11 - step by step 5
5. Ubuntu 22.04 / 24.04 - step by step 6
6. macOS 13+ - step by step 7
7. Using the running system 8
7.1 Submit a TMF641 order 8
7.2 Tenants 8
7.3 Logs, data and results 8
7.4 The DxPS console 9
7.5 Regenerate the test-case document 10
8. Configuration reference 10
9. Troubleshooting 10
10. Security notes 10
11. Uninstall 11

1. What you are installing

DxPS is a multi-tenant real-time provisioning system for MNOs and MVNOs. It takes TM Forum TMF641 service orders, plans them into network element (NE) tasks, executes them with sagas over Kafka and reports progress with TMF688 events. The package runs the complete system on one machine: PostgreSQL 18, Apache Kafka 4, six DxPS services, NE simulators for every southbound interface, and the Mosaic live dashboard.

Component	Address (loopback only)	Purpose
Mosaic dashboard	http://127.0.0.1:8088	Live view: tenants, lanes, topics, orders, NEs, events, tests; load generator and attack probes
DxPS console (web UI)	https://localhost:8092	Signed-in console for operations and configuration: orders, DLQ, simulators, catalog studio, Template Lab, tenants, NE inventory, reports, users (optional, Ubuntu and macOS)
Console API	https://localhost:8446	Back end of the console: sessions and roles, cross-tenant views, inventory edits, catalog drafts, audit
Gateway (TMF641/TMF688 hub)	https://localhost:8443	Northbound API, JWT auth, per-tenant rate limits
Orchestrator	127.0.0.1:9201 (stats)	Plans orders, runs sagas, dependencies and per-subscriber sequencing
Adapter	127.0.0.1:9202 (stats)	Southbound drivers: 5G SBA, IMS, NETCONF/USP, eSIM, BSS/OCS, NPDB, CAMARA
Event hub	127.0.0.1:9203 (stats)	Signed TMF688 webhooks with retries
NE simulators	9101-9108 (mTLS), admin 9199, webhook sink 9109	NRF, UDR, IMS, RESTCONF/USP, SM-DP+, OCS/BSS, NPDB, NEF/CAMARA
PostgreSQL 18	localhost:5433 (TLS 1.3, SCRAM)	Orders, plans, tasks, outbox, audit; row-level security per tenant
Apache Kafka 4 (KRaft)	127.0.0.1:9092	41 topics: business commands, NE tasks, retry ladder, events, DLQ

Note: Every listener binds to the loopback interface. Nothing is exposed to the network.

2. Requirements

Item	Minimum	Recommended
CPU	4 cores (x86-64 or ARM64)	8 cores
Memory	8 GB	16 GB
Disk	6 GB free (tools, data, logs)	15 GB
Windows	Windows 10 22H2 / Windows 11, x64, PowerShell 5.1+, winget	Windows 11
Ubuntu	22.04 LTS or 24.04 LTS, a normal user with sudo	24.04 LTS
macOS	macOS 13 Ventura or later, Homebrew, Xcode Command Line Tools	macOS 14+ on Apple silicon
Network	HTTPS to go.dev/dl.google.com, downloads.apache.org (or archive.apache.org), apt.postgresql.org / Homebrew / get.enterprisedb.com, proxy.golang.org	
Free ports	5433, 8088, 8443, 9092, 9093, 9101-9109, 9199, 9201-9203 (console: 8092, 8446)	
Node.js (console only)	22.12 or later with npm	Node.js 22 LTS

The installer scripts fetch these versions (all can be overridden, see section 8):

Software	Windows	Ubuntu	macOS
Go 1.26.6	winget GoLang.Go	official tarball, SHA-256 verified, into the runtime	Homebrew go
Java 21 (Kafka needs 17+)	winget Eclipse Temurin 21 JRE	apt openjdk-21-jre-headless	Homebrew openjdk@21
PostgreSQL 18	EDB binaries zip, into the runtime	apt.postgresql.org postgresql-18	Homebrew postgresql@18
Apache Kafka 4.3.1	Apache release, SHA-512 verified	Apache release, SHA-512 verified	Apache release, SHA-512 verified

3. Package contents

The release (GitHub release v<version>, or the DxPS_Delivery_<date> folder) contains:

```
dxps-<version>-windows.zip        Windows bundle (source, scripts, manuals)
dxps-<version>-ubuntu.tar.gz      Ubuntu / macOS bundle (same content; scripts keep their execute bit)
dxps-<version>-evidence.zip       test evidence: tests.json, coverage, probes, govulncheck, gosec, payload
traces
SHA256SUMS.txt                    checksums of the files above

dxps/ (inside each bundle)
  RELEASE.txt  start here: contents and quick start for the platform
  cmd/         gateway orchestrator adapter eventhub netsim dashboard console dxpsctl
  internal/    packages (catalog, planner, saga, bus, store, console, ...) with their tests
  console/     DxPS console web UI (Next.js) and its tests
  scripts/     *.ps1 for Windows, *.sh for Ubuntu and macOS
  docs/manuals/  HLD+LLD, Test Cases and Results, Components and Data Guide, this guide (docx + pdf)
  INSTALL.md   this guide in Markdown
```

All scripts keep their state in one runtime directory, by default %USERPROFILE%\dxps-runtime on Windows and ~/dxps-runtime on Ubuntu and macOS (override with DXPS_RUNTIME). The source tree is never modified.

Script (Windows .ps1 / Ubuntu+macOS .sh)	What it does	When
install-deps-windows / -ubuntu / -macos	Installs Go, Java, PostgreSQL 18 and Kafka 4	once
infra-setup	Generates secrets and the dev PKI, creates the PostgreSQL cluster, roles and database, formats Kafka	once
run-all	Starts PostgreSQL and Kafka, builds all services, migrates, creates topics, seeds reference data, starts the stack	every start
run-all -NoBuild / --no-build	Same without rebuilding	fast restart
status	Shows which components are listening	any time
test-all	Runs all unit, integration and E2E tests with coverage; publishes results to the Mosaic Tests tile	after changes
sec-scan	govulncheck + gosec static security scans	before delivery
infra-stop	Stops services, Kafka and PostgreSQL	end of day
infra-start	Starts only PostgreSQL and Kafka	for development
console-run (.sh only)	Builds and starts the console API and web UI; needs the platform running and Node.js 22.12+	to use the console
console-stop (.sh only)	Stops the console API and web UI	any time

4. Windows 10/11 - step by step

Step 1 Unpack the source

Extract dxps-<version>-windows.zip, for example to C:\ (it creates C:\dxps). Open Windows PowerShell in that folder (the folder that contains go.mod) and allow local scripts for this window only.

```
cd C:\dxps
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Step 2 Install the prerequisites

Installs Go and Java with winget if missing, and downloads PostgreSQL 18 binaries and Kafka 4 into the runtime directory. When winget installs Go or Java, close PowerShell, open a new window and repeat the command so PATH is refreshed.

```
.\scripts\install-deps-windows.ps1
```

Step 3 Create the runtime

Generates secrets.env (random passwords and keys, never printed), the dev CA and service certificates, initialises PostgreSQL on port 5433 with TLS 1.3 and SCRAM, creates the roles dxps_owner, dxps_app (row-level security enforced) and dxps_ops, and formats Kafka in KRaft mode.

```
.\scripts\infra-setup.ps1
```

Step 4 Build and start everything

Takes 1-3 minutes the first time (Go module download and build).

```
.\scripts\run-all.ps1
```

Step 5 Open the Mosaic

Browse to http://127.0.0.1:8088. Press Start load to send a realistic order mix, Run probes to attack the live system with 24 security probes, and use the chaos slider to inject NE faults.

Step 6 Check and test

```
.\scripts\status.ps1
.\scripts\test-all.ps1      # ~2 minutes, results appear in the Tests tile
.\scripts\sec-scan.ps1
```

Step 7 Stop and restart

```
.\scripts\infra-stop.ps1
.\scripts\run-all.ps1 -NoBuild
```

Note: Windows-specific: Kafka on Windows cannot rename open log segments (KAFKA-1194), so the dev broker runs with log compaction and retention deletion disabled. Do not delete Kafka topics on Windows; it takes the broker down. Linux and macOS use Kafka's defaults.

5. Ubuntu 22.04 / 24.04 - step by step

Step 1 Unpack the source

Use a normal user with sudo rights, not root: PostgreSQL refuses to run as root.

```
mkdir -p ~/dxps && tar -xzf dxps-<version>-ubuntu.tar.gz -C ~/dxps --strip-components=1
cd ~/dxps
```

Step 2 Install the prerequisites

Adds the official PostgreSQL apt repository and installs postgresql-18 without creating the system cluster, installs OpenJDK 21, installs Go 1.26.6 into the runtime if the system Go is older, and downloads Kafka 4. Asks for your sudo password.

```
bash scripts/install-deps-ubuntu.sh
```

Step 3 Create the runtime

Same as on Windows: secrets, PKI, PostgreSQL cluster on 5433 with TLS 1.3 + SCRAM, roles, database, Kafka KRaft storage.

```
bash scripts/infra-setup.sh
```

Step 4 Build and start everything

```
bash scripts/run-all.sh
```

Step 5 Open the Mosaic

Browse to http://127.0.0.1:8088. On a remote server, tunnel it instead of exposing it: ssh -L 8088:127.0.0.1:8088 user@server, then open http://127.0.0.1:8088 locally.

Step 6 Check and test

```
bash scripts/status.sh
bash scripts/test-all.sh
bash scripts/sec-scan.sh
```

Step 7 Stop and restart

```
bash scripts/infra-stop.sh
bash scripts/run-all.sh --no-build
```

6. macOS 13+ - step by step

Step 1 Install Homebrew and the command line tools

Skip if you already have them.

```
xcode-select --install
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Step 2 Unpack the source

Use the Ubuntu bundle; it is the same for macOS.

```
mkdir -p ~/dxps && tar -xzf dxps-<version>-ubuntu.tar.gz -C ~/dxps --strip-components=1
cd ~/dxps
```

Step 3 Install the prerequisites

Installs go, postgresql@18 and openjdk@21 with Homebrew (no brew services are started) and downloads Kafka 4 into the runtime.

```
bash scripts/install-deps-macos.sh
```

Step 4 Create the runtime

```
bash scripts/infra-setup.sh
```

Step 5 Build and start everything

macOS may ask once whether the services may accept incoming connections. They only listen on loopback; Allow or Deny both work for local use.

```
bash scripts/run-all.sh
```

Step 6 Open the Mosaic

Browse to http://127.0.0.1:8088.

Step 7 Check and test

```
bash scripts/status.sh
bash scripts/test-all.sh
bash scripts/sec-scan.sh
```

Step 8 Stop and restart

```
bash scripts/infra-stop.sh
bash scripts/run-all.sh --no-build
```

7. Using the running system

7.1 Submit a TMF641 order

Save this order as order.json. Subscriber numbers ending in 997, 998 or 999 trigger simulator faults (transient, permanent rejection, unavailable), so this example avoids them.

```
{
  "@type": "ServiceOrder",
  "externalId": "DEMO-0001",
  "channel": [{"name": "SELFCARE"}],
  "serviceOrderItem": [{
    "id": "1", "action": "add",
    "service": {
      "serviceSpecification": {"id": "CreateSubscriber5G"},
      "serviceCharacteristic": [
        {"name": "supi",   "value": "imsi-416770000004321"},
        {"name": "msisdn", "value": "962790004321"},
        {"name": "plan",   "value": "5G-100GB"}
      ]
    }
  }]
}
```

Easiest on every platform: dxpsctl api mints a short-lived token for the tenant internally (never printed) and calls the gateway over TLS with the dev CA.

```
# Ubuntu / macOS
~/dxps-runtime/bin/dxpsctl api -tenant mvno-alpha -idem demo-0001 -body "$(cat order.json)" POST /serviceOrder
~/dxps-runtime/bin/dxpsctl api -tenant mvno-alpha GET "/serviceOrder?limit=5"

# Windows
$ctl = "$env:USERPROFILE\dxps-runtime\bin\dxpsctl.exe"
& $ctl api -tenant mvno-alpha -idem demo-0001 -body (Get-Content -Raw order.json) POST /serviceOrder
& $ctl api -tenant mvno-alpha GET "/serviceOrder?limit=5"
```

With curl (Ubuntu/macOS). The token is a credential: keep it in a variable, do not paste it into tickets or chat.

```
RT=~/dxps-runtime
TOKEN=$($RT/bin/dxpsctl token -tenant mvno-alpha -ttl 10m)
curl --cacert $RT/pki/ca.crt -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
     -H 'Idempotency-Key: demo-0002' --data @order.json \
     https://localhost:8443/tmf-api/serviceOrdering/v5/serviceOrder
unset TOKEN
```

The gateway answers 201 with the order id and state acknowledged. Sending the same Idempotency-Key again returns 200 with Idempotent-Replayed: true. Follow the order in the Mosaic Orders and Events tiles.

7.2 Tenants

Tenant	Type	Notes
host-mno	Host MNO	owns the shared core network
mvno-alpha	Full MVNO	own OCS and a tenant-specific CreateSubscriber5G variant
mvno-beta	Light MVNO	uses the host's shared NEs
ent-acme	Enterprise	enterprise customer of the host MNO
mvno-gamma	Light MVNO	suspended: every call is refused (403 DXPS-1004)

7.3 Logs, data and results

Path (under the runtime directory)	Content
logs/svc/<service>.log	JSON logs of each DxPS service (no secrets)
logs/postgres.log, logs/kafka/server.log	PostgreSQL and Kafka logs
results/tests.json, coverage.out, tests-raw.jsonl	latest test-all run

results/govulncheck.txt, gosec.json	latest sec-scan run
secrets.env (mode 0600)	generated passwords, DSNs and keys - never share or commit
pki/	dev CA and certificates (development only)
data/pg, data/kafka	PostgreSQL cluster and Kafka log directories
console/users.json (mode 0600)	console accounts: PBKDF2-SHA256 password hashes and roles
console/drafts/, console/audit.jsonl	catalog drafts and the console's own audit trail
logs/svc/console.out, console-ui.out	console API and web UI start-up output

7.4 The DxPS console

The console is a signed-in web application for running and configuring the platform. It reads every tenant's orders, tasks and inventory, shows Kafka, dead letters and the simulators live, and lets authorised users edit tenants, NE limits and entitlements, work on catalog drafts, dry-run them in the Template Lab and submit real orders through the gateway. It runs on Ubuntu and macOS; there is no Windows script for it yet.

Step 1 Install Node.js 22

Needed only for the console. macOS: brew install node@22 (console-run finds it automatically). Ubuntu: install Node.js 22 LTS from NodeSource or nvm so that node and npm are on PATH.

```
node --version   # v22.12 or later
```

Step 2 Start the console

Run this after run-all. It builds the console API and the web UI, starts both detached and issues the console certificate from the dev CA on first start. Add --foreground to keep it attached.

```
bash scripts/console-run.sh
```

Step 3 Create the first user

Console accounts are separate from tenants. The password (12 characters or more) is read from standard input, or from DXPS_CONSOLE_PASSWORD.

```
~/dxps-runtime/bin/console useradd -name admin -roles admin
```

Step 4 Sign in

Browse to https://localhost:8092 and sign in. Create further accounts with the right roles on the Users page. On a remote server, tunnel it: ssh -L 8092:127.0.0.1:8092 user@server.

Step 5 Stop

```
bash scripts/console-stop.sh
```

Role	Can
viewer	See every page
operator	Also submit and cancel orders, inject NE faults, redrive dead letters, start load runs and probes
configurator	Also edit tenants, NE limits and entitlements, work on catalog drafts and use the Template Lab
admin	Also manage console users

Area	Pages
Operations	Dashboard, Service orders (task graph, NE requests and responses, attempts, cancel), NE tasks, Dead letters, Network and simulators (fault injection), Kafka
Configuration	Service catalog, Catalog studio, Template Lab, Order composer, Tenants, NE inventory
Insight	Reports, Load and probes, Test results, Webhooks, Audit trail
Administration	Users, Settings, My account

Changing the catalog: the services run with the catalog compiled in, so catalog changes are made as drafts. Create a draft in the Catalog studio (or with Create override draft on a spec), edit it, Validate it (the whole catalog is loaded with the change and each spec's example is run through the Lab), try it in the Template Lab, then Export it, copy the files into dxps/internal/catalog/data and restart with run-all.

Note: Tenant, NE limit and entitlement edits apply live within about 10 seconds and are written to the DxPS audit trail with actor console:<user>. run-all re-seeds the demo tenants and inventory when it starts, so those edits return to the seed values after a restart.

7.5 Regenerate the test-case document

After test-all, the Word report can be rebuilt from the new results with Python 3.10+, python-docx and matplotlib (design/build_dxps_testcases_docx.py in the delivery workspace). On Windows, Word exports the PDF; on Ubuntu/macOS use LibreOffice: soffice --headless --convert-to pdf DxPS_Test_Cases_and_Results.docx.

8. Configuration reference

Variable	Default	Meaning
DXPS_RUNTIME	~/dxps-runtime (Windows: %USERPROFILE%\dxps-runtime)	runtime directory for tools, data, logs, secrets
DXPS_PGBIN	Ubuntu /usr/lib/postgresql/18/bin; macOS $(brew --prefix postgresql@18)/bin	PostgreSQL binaries (Ubuntu/macOS scripts)
DXPS_PG_MAJOR	18	PostgreSQL major version for the Ubuntu/macOS installers
DXPS_KAFKA_VERSION	4.3.1	Kafka release downloaded by the installers (Windows: -KafkaVersion)
DXPS_GO_VERSION	1.26.6	Go version installed on Ubuntu when the system Go is older
KAFKA_HEAP_OPTS	-Xms512m -Xmx1g	Kafka JVM heap
DXPS_NETSIM_HOST	localhost	host name the seed uses for simulator endpoints
HTTPS_PROXY / GOPROXY	-	corporate proxy for downloads and Go modules

9. Troubleshooting

Symptom	Cause and fix
'port NNNN did not open'	Another program uses the port, or the component failed: run status, then read the component's log (section 7.3). On Linux: ss -ltnp | grep NNNN; macOS: lsof -iTCP:NNNN -sTCP:LISTEN.
Kafka does not start	java -version must be 17 or later. Read logs/kafka/server.log. On macOS check that openjdk@21 is installed (the scripts set JAVA_HOME automatically).
Windows: Kafka stops with 'all log dirs have failed'	A topic was deleted or segments were renamed. Stop the stack, delete only the partition folders named in server.log, start again.
'initdb: cannot be run as root'	Run the Ubuntu/macOS scripts as a normal user with sudo rights.
Ubuntu: 'could not create lock file /var/run/postgresql/...'	The cluster tried to use the system socket directory. Re-run infra-setup: it sets unix_socket_directories = '' (DxPS uses TLS over TCP only).
PostgreSQL 'could not load server certificate'	Certificate permissions changed. The files in data/pg must be owned by your user with mode 0600; re-copy them from pki/.
macOS: postgresql@18 not found	brew update && brew install postgresql@18, or set DXPS_PGBIN to another PostgreSQL 18 bin directory.
Go build downloads fail	Set HTTPS_PROXY (and GOPROXY if your company mirrors modules).
Dashboard shows a service as down	Restart with run-all; logs/svc/<service>.out shows start-up errors.
console-run: 'Node.js 22.12+ is required'	Install Node.js 22 (section 7.4) and open a new terminal so node is on PATH.
console-run: 'UI build failed'	Read logs/svc/console-ui-build.out. Without network access to the npm registry, the first build cannot install the UI packages.
Console sign-in fails with the right password	Five wrong passwords in a row lock the account until an admin resets its password on the Users page. A disabled account cannot sign in either.
Console pages show 'console API unavailable'	The console API is not running: start it again with console-run and read logs/svc/console.out.
Start from scratch	Stop the stack, then remove the runtime directory (this deletes all data and secrets) and rerun infra-setup.

10. Security notes

- Secrets are generated locally with crypto/rand, stored only in secrets.env (mode 0600) and never printed by the scripts.

- PostgreSQL accepts only TLS 1.3 + SCRAM-SHA-256 on loopback; the application role is subject to row-level security.
- All NE simulator links use mutual TLS; the gateway uses TLS with the dev CA; webhooks are HMAC-signed.
- The console API listens on loopback only and the web UI is served over TLS. The session token is kept in an HttpOnly, SameSite=Strict, Secure cookie that page scripts cannot read, and every change needs a CSRF header. Orders from the console go through the gateway with a 2-minute token limited to order read and write.
- The dev CA, PLAINTEXT Kafka on loopback and the single-node layout are for development and demonstration. Production uses the HLD/LLD deployment (Kafka SASL_SSL, managed PKI/HSM, Kubernetes).

11. Uninstall

- Stop the stack with infra-stop, then delete the runtime directory and the source folder.
- Windows: winget uninstall GoLang.Go / EclipseAdoptium.Temurin.21.JRE (only if they were installed for DxPS).
- Ubuntu: sudo apt remove postgresql-18 openjdk-21-jre-headless; sudo rm /etc/apt/sources.list.d/pgdg.list.
- macOS: brew uninstall postgresql@18 openjdk@21 go.